Trabajo Final - Coloquio

Informatica y programacion 2026

Estación de monitoreo de temperatura



MATAS SOTANA Nahuel Carlos 14757
42862695
Grupo nº1 Individual

Trabajo Final - Coloquio **1**
introducción general **4**
Parte A **5**
Introducción 6
Requerimientos y restricciones 7
Circuito utilizado 8
Uso del Sensor BMP 280 medición de Temperatura 8
Código y configuración del Esp 32 9
Algoritmo General de Alto Nivel de Abstracción python 11
Diagrama de Flujo de la solución Pc Estacion 12
Código en python 13
Librerías utilizadas: 13
1. Librería serial (PySerial) **13**
2. Librería time 13
3. Librería threading **14**
4. Librería datetime **14**
Variables Globales y Eventos de Sincronización 15
Configuración Inicial del ESP32 (Puerto Serie) 16
Descripción de las subrutinas: 16
1. Boton_temporizador_Parpadeo(mensaje) 16
2. parpadeo() 17
3. temporizador_temperatura() 17
4.procesar_temperatura(valor_str) 18
5.guardar_en_archivo(temp_actual, tendencia) 19
6.bucle_principal() (Explicación) 19
Explicación del funcionamiento del código con capturas de pantalla 21
Parte B **23**
Introducción 23
¿Por qué utilizar la desviación estándar? 23
Requerimientos y Restricciones 23
Algoritmo General de Alto Nivel de Abstracción 24
Librerías utilizadas 25
1. Librería <stdio.h> (Standard Input/Output) **25**
2. Librería <stdlib.h> (Standard Library) **25**
3. Librería <math.h> (Math) **25**
4. Librería <string.h> (String) **26**
Subrutinas creadas 26
comparar_temperaturas() **26**
Código C 27
Captura de pantalla demostrando el funcionamiento 29
Parte C **30**
Introducción 30
Requerimientos y restricciones específicas 30

Librerias adicionales utilizadas....31
Librería socket....31
Configuración de Red (Sockets TCP/IP)....32
Algoritmo General de Alto Nivel de Abstracción....33
Código Python Pc principal....34
Captura de pantalla demostrando el funcionamiento....36
Conclusión final....37

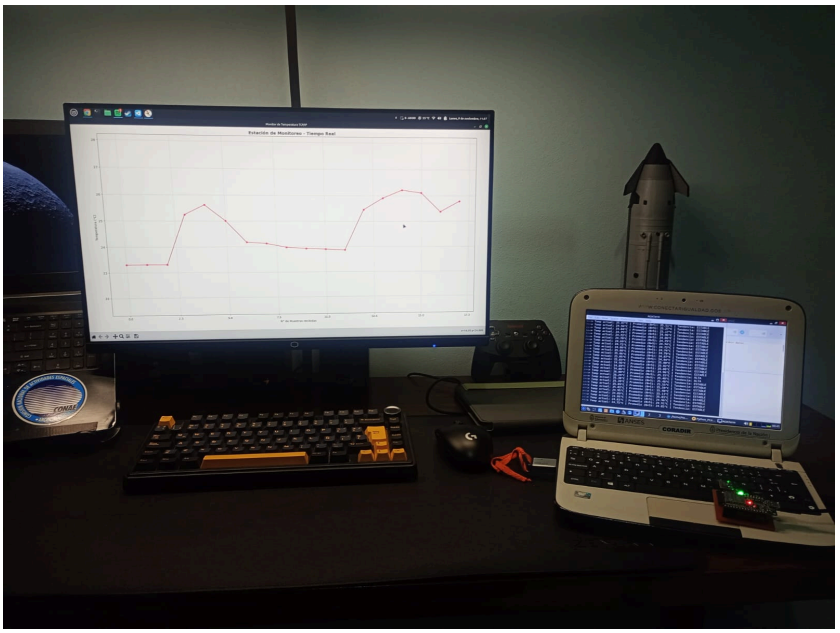
introducción general

Este proyecto consiste en el desarrollo de un sistema de monitoreo de temperatura, diseñado de manera tal que se implementen distintas etapas haciendo énfasis en los temas y herramientas vistos durante el cursado.

Para lograrlo y organizar el flujo de los datos, el trabajo se dividió en tres grandes etapas que funcionan en cadena:

1. **Adquisición de datos (Hardware y Control):** El ESP32 interactúa con el entorno físico. Conectado por USB a una computadora, el microcontrolador obedece las órdenes de un programa "maestro" escrito en Python. Este programa se encarga de pedir las lecturas de temperatura, controlar luces LED, calcular la tendencia del clima y guardar todo este historial en un archivo de texto local.
2. **Procesamiento Estadístico (Análisis en C):** Una vez que los datos están guardados, entra en juego un programa desarrollado en lenguaje C. Su trabajo es leer ese archivo histórico de forma eficiente y calcular estadísticas clave (como promedios, valores máximos, mínimos y la desviación estándar). Esto permite tener un diagnóstico de cómo se comportó el ambiente a lo largo del tiempo.
3. **Visualización Remota (Redes y Gráficos):** Se implementó un segundo módulo en Python que transmite los datos a través de una red local (usando TCP/IP). En el otro extremo, una computadora recibe esta información y dibuja la curva de temperatura en un gráfico animado en tiempo real.

Para cumplir con los objetivos físicamente, el hardware del proyecto se compone de dos PC: una computadora principal, donde se mostrará la gráfica de la temperatura en el tiempo y funcionará como servidor, y una computadora secundaria denominada "PC estación". Esta última es una computadora portátil conectada a la interfaz del ESP32, el cual cuenta con un sensor, múltiples LEDs y un botón físico.



Parte A

- Se plantea realizar una estación de monitoreo y seguimiento de temperatura. La cual debe cumplir con los siguientes requisitos:
  - Debe realizar la lectura de temperatura en ciclos de X segundos. Por defecto el ciclo es de 3.5 segundos. Al iniciar, el equipo debe comenzar a registrar datos con esta frecuencia.
  - En cada ciclo la estación muestra por consola la temperatura que representa la última temperatura leída y el promedio de los últimos N de los eventos, (por defecto N=5).
  - El ciclo se configura mediante un pulsador: al presionarlo, comienza el conteo en milisegundos y, al soltarlo, se finaliza. Se notificará al usuario el cambio en la configuración; el equipo comenzará a registrar datos inmediatamente a la nueva frecuencia. El ciclo mínimo es de 2.5 segundos (pulsando desde 1" hasta 2.5") y el ciclo máximo es de 10 (pulsando desde 10" en adelante). Una vez concluído el cálculo del ciclo se informa por pantalla el nuevo tiempo del ciclo y se registra a partir de este momento mediante la nueva frecuencia.
  - El mismo pulsador se utiliza para finalizar el programa de monitoreo, esto es, si el pulsador se mantiene apretado por menos de 1 segundo.
  - La estación además posee un semáforo con 3 leds los cuales indican la tendencia de cambio de la temperatura, es decir,
    - Un led rojo se enciende cuando el valor de la lectura en un ciclo supera en un X% del promedio de los últimos N eventos (tendencia en alta).
    - Un led amarillo indica que la temperatura se mantiene estable, es decir se encuentra dentro del promedio (+-X%)
    - Un led verde enciende cuando la temperatura disminuye en un X% del promedio de los últimos N eventos (tendencia en baja).
    - Cada vez que un led enciende, los otros dos se apagan.
    - Los 3 leds de tendencia se mantendrán prendidos cuando **no** se tenga la suficiente información para analizar la tendencia. Esto es así hasta que se dispongan de al menos el N eventos necesarios para calcular el promedio.
    - Mientras el pulsador esté presionado, una vez por segundo se deberán destellar durante 50ms todos los leds indicando los segundos, luego se prenderá el led correspondiente a la tendencia actual o todos si no la hubiere.
    - Mientras la estación esté midiendo temperaturas se deberán destellar durante 50ms todos los leds indicando el fin del ciclo y la captura realizada, luego se prenderá el led correspondiente a la tendencia actual o todos si no la hubiere.
  - Una última tarea que realiza esta estación de monitoreo es ir registrando en un archivo por cada ciclo:

- Lectura de temperatura
- Tendencia detectada (para cada uno de los 4 estados posibles)
- Fecha y Hora en que se realizó la lectura

Sobre la resolución:

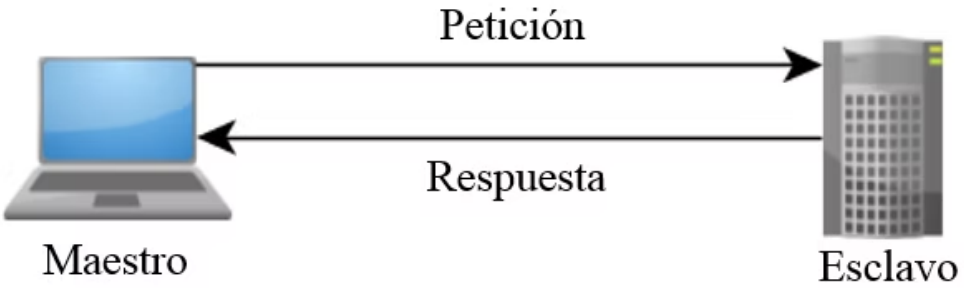
- Se debe realizar un análisis y diseño algorítmico previo a su implementación. Identificar: entradas, salidas, información relevante, requerimientos y restricciones. Además elaborar un algoritmo general de alto nivel de abstracción que resuelva el problema.
- El ejercicio debe ser codificado en Python.

Introducción

La primera etapa de este proyecto (Parte A) abarca el diseño e implementación del firmware para el microcontrolador ESP32, el cual funciona como la interfaz física entre el entorno real y el sistema de software de la computadora.

Para esta fase, el ESP32 fue programado para operar bajo una comunicación "maestro-esclavo", donde el script de Python en la PC asume el rol de maestro. Toda la transferencia de datos y recepción de comandos se establece de forma bidireccional a través del puerto serie nativo del microcontrolador.

Por otro lado, este Maestro se ejecuta en una computadora conectada por cable USB al microcontrolador. El programa tiene la capacidad de emitir órdenes precisas hacia el ESP32. Esto incluye el control de los indicadores visuales (encendido y apagado de LEDs, de forma individual o en conjunto) y la emisión de solicitudes de lectura (peticiones de medición de temperatura) cuando el sistema lo requiere. Simultáneamente, el código monitorea el puerto serie para capturar las respuestas del microcontrolador, encargándose de recibir, decodificar y registrar los valores medidos. Luego, el programa calcula la tendencia y almacena todo el registro de lecturas localmente en la propia computadora.



Requerimientos y restricciones

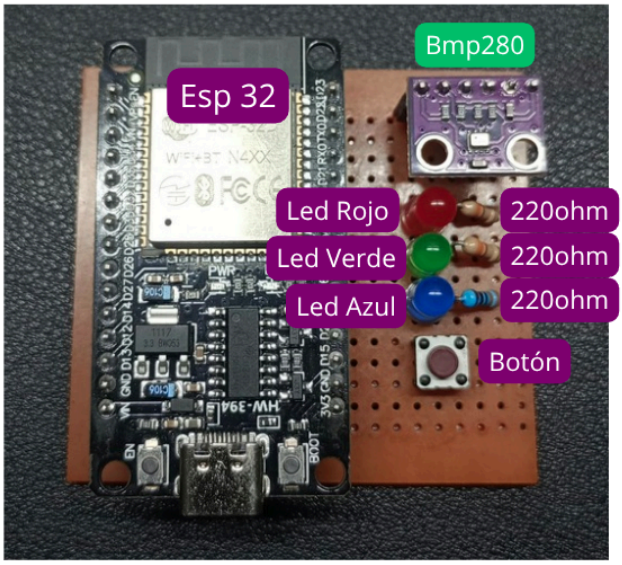
Previo a la elaboración del código, se realizó un análisis del problema a resolver para identificar los elementos necesarios,posibles requerimientos y restricciones.

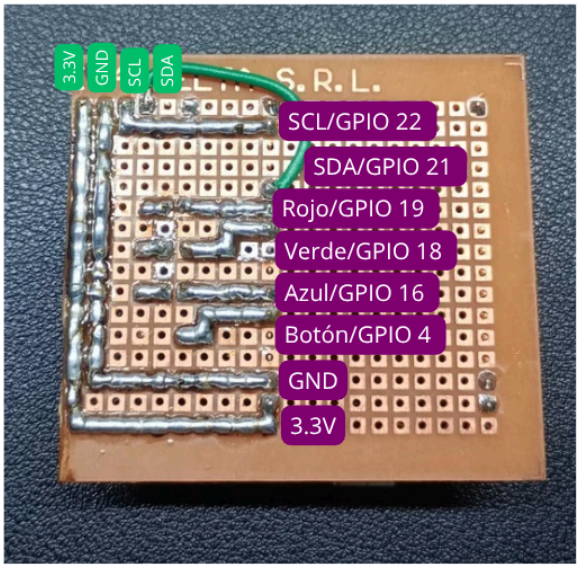
Identificación de Elementos

- Entradas (Inputs):
  - Cadenas de texto que indican el estado del boton: "BOTON_ON" (presionado) y "BOTON_OFF" (soltado).
  - Cadenas de texto terminadas en el carácter 'c' que representan la lectura térmica
- Salidas (Outputs):
  - Puerto Serie: Comandos de control enviados al ESP32 para accionar los LEDs (`'E'`, `'X'`, `'V'`, `'R'`, `'A'`, `'T'`).
  - Almacenamiento Local: Cadenas de texto con la fecha, hora, temperatura y tendencia en un archivo local
- Variables de Estado y Constantes:
  - Intervalo de muestreo actual (`ultimo_tiempo_guardado`).
  - Tiempo transcurrido durante la pulsación del botón (Delta T).
  - Historial de las últimas 5 mediciones..
  - Banderas lógicas que indican el estado del sistema.
- Restricciones:
  - El tiempo de lectura no puede ser inferior a 2.5 segundos ni superior a 10.0 segundos, a menos que el sistema entre en pausa (por un clic menor a 1 segundo).
  - El algoritmo de tendencia requiere un volumen mínimo de datos. Hasta no acumular 5 mediciones, el sistema no puede emitir una tendencia y debe mantener el estado "INICIAL".
  - No se puede realizar peticiones de temperatura mientras el botón se encuentre presionados
  - El programa debe ser capaz de escuchar el puerto serie ininterrumpidamente mientras gestiona rutinas temporizadas (parpadeo y peticiones de datos) en paralelo.

Circuito utilizado

El circuito implementado está conformado por un microcontrolador ESP32, un sensor de presión y temperatura BMP280, tres LEDs que indican la tendencia actual (indicadores luminosos de distintos colores) con sus correspondientes resistencias limitadoras de corriente de 220 Ω y un botón pulsador que permite el ajuste del tiempo de muestreo. Todos los componentes se encuentran montados y soldados sobre una placa de circuito impreso perforada (PCB experimental), lo que evita falsos contactos o desconexiones accidentales y mejora la presentación.

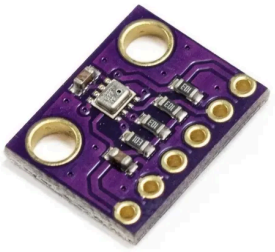


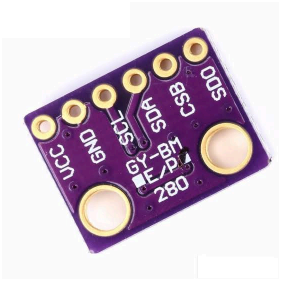


Uso del Sensor BMP 280 medición de Temperatura

Para conocer la temperatura del ambiente en este proyecto, se eligió el sensor BMP280.Aunque se trata de un componente frecuentemente empleado para la medición de la presión barométrica , también incluye internamente un sensor de temperatura para la corrección de la medición del barómetro ,lo que lo hace ideal para registrar los cambios de temperatura en nuestro entorno de forma constante.

El módulo opera con un voltaje de 3.3 V, lo cual facilita una conexión directa con el microcontrolador ESP32. La transmisión de datos se realiza mediante el protocolo I2C, el cual permite la interconexión de múltiples sensores compartiendo únicamente dos líneas (SDA y SCL), asignando a cada dispositivo una dirección interna propia.Para ello se utilizó en el Esp 32 la librería Adafruit_BMP280.h





Parámetros del sensor:

Eléctricas y de Comunicación

- Voltaje de operación: 1.71 V a 3.6 V (ideal para la lógica de 3.3 V del ESP32).
- Consumo de energía: Extremadamente bajo; consume alrededor de 2.7 µA midiendo a 1 Hz y apenas 0.1 µA en modo de reposo (sleep mode).
- Interfaces de datos: Soporta I2C (hasta 3.4 MHz) y SPI (de 3 o 4 hilos, hasta 10 MHz).
- Direcciones I2C: Típicamente `0x76`

Medición de Temperatura

- Rango de operación: -40 °C a +85 °C.
- Precisión absoluta: ±1.0 °C (dentro del rango de 0 °C a +65 °C).
- Resolución: 0.01 °C.

Medición de Presión (Barómetro/Altímetro)

- Rango de medición: 300 a 1100 hPa.
- Rango de altitud equivalente: Desde +9000 metros hasta -500 metros sobre el nivel del mar.
- Precisión absoluta: ±1 hPa (lo que se traduce en un error de cálculo de altitud de aproximadamente ±1 metro).
- Precisión relativa: ±0.12 hPa (resolución equivalente a ±1 metro para medir variaciones o escalones de altura).

Código y configuración del Esp 32

En la configuración actual, el ESP32 actúa como un esclavo del código de Python. Este tiene dos modos de operación: por un lado, permanece a la escucha de las órdenes de Python, las cuales pueden ser encender o apagar uno o todos los LEDs, o realizar una medición de temperatura para luego responder. Por otro lado, el ESP32 puede indicarle al programa que el botón fue presionado o soltado. La comunicación bidireccional se realiza mediante el puerto serie del propio ESP32.

```ino
#include <Wire.h>
#include <Adafruit_BMP280.h>

Adafruit_BMP280 bmp;

// --- Pines ---
const int boton = 4;
const int azul = 16;
const int verde = 18;
const int rojo = 19;

// --- Variables ---
int estadoAnteriorBoton = LOW;

void setup() {
  Serial.begin(115200);

  pinMode(boton, INPUT_PULLUP);
  pinMode(azul, OUTPUT);
  pinMode(verde, OUTPUT);
  pinMode(rojo, OUTPUT);

  // Apagar los LEDs por defecto al iniciar
  digitalWrite(azul, LOW);
  digitalWrite(verde, LOW);
  digitalWrite(rojo, LOW);

  if (!bmp.begin(0x76)) {
    Serial.println("ERROR: BMP280");
    while (1);
  }
}
```

```ino
void loop() {
  // ========================================
  // 1. ESCUCHAR ORDENES
  // ========================================
  if (Serial.available() > 0) {
    char comando = Serial.read();

    // LEDs individuales: Invierten el estado actual (!digitalRead)
    if (comando == 'A') digitalWrite(azul, !digitalRead(azul));
    if (comando == 'V') digitalWrite(verde, !digitalRead(verde));
    if (comando == 'R') digitalWrite(rojo, !digitalRead(rojo));

    // TODOS los LEDS
    if (comando == 'E') { // Encender todos
      digitalWrite(azul, HIGH);
      digitalWrite(verde, HIGH);
      digitalWrite(rojo, HIGH);
    }
    if (comando == 'X') { // Apagar todos
      digitalWrite(azul, LOW);
      digitalWrite(verde, LOW);
      digitalWrite(rojo, LOW);
    }

    // SENSOR: Si Python envía la 'T', el ESP32 lee y devuelve la temperatura
    if (comando == 'T') {
      float temperatura = bmp.readTemperature();

      // envía el número seguido de la letra 'c'
      Serial.print(temperatura);
      Serial.println("c");
    }
  }
```

```ino
  // ========================================
  // 2. ENVIAR ESTADO DEL BOTÓN
  // ========================================
  int estadoBoton = !digitalRead(boton);

  if (estadoBoton == HIGH && estadoAnteriorBoton == LOW) {
    Serial.println("BOTON_ON");
  }
  else if (estadoBoton == LOW && estadoAnteriorBoton == HIGH) {
    Serial.println("BOTON_OFF");
  }
  estadoAnteriorBoton = estadoBoton;

  // Antirrebote
  delay(10);
}
```

Algoritmo General de Alto Nivel de Abstracción python

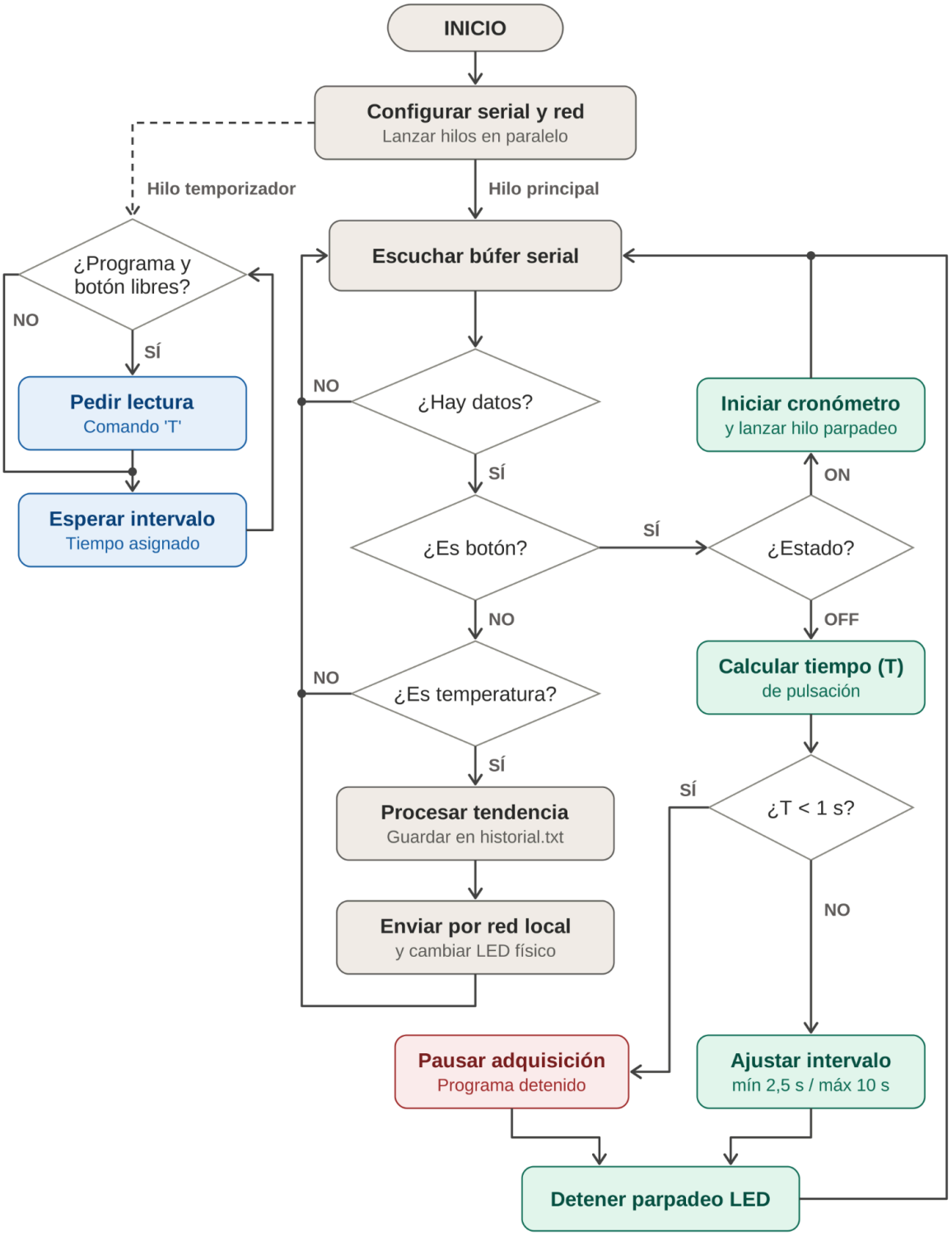
El siguiente algoritmo describe la lógica del flujo principal del programa python de forma general.

1. **Inicialización de interfaces:**
    - Abrir la conexión Serial con el ESP32.
    - Intentar abrir la conexión TCP con el servidor remoto.
2. **BUCLE PRINCIPAL :**

Mientas el programa esté en ejecución:

- Verificar continuamente el estado del programa. Si el programa NO está detenido y el botón NO está presionado, enviar comando de solicitud de temperatura (' T ') y dormir durante el intervalo actual.
    - Leer el búfer del puerto serie.
    - SI el mensaje es "BOTON_ON":
        - Registrar el tiempo de inicio.
        - Activar bandera de *botón presionado*.
        - Hacer parpadear los leds
    - SI el mensaje es "BOTON_OFF":
        - Desactivar bandera de *botón presionado*.
        - Detener el parpadeo
        - Calcular el tiempo total presionado.
        - SI tiempo < 1s: Pausar el programa.
        - SINO: Ajustar el intervalo de muestreo (mínimo 2.5s, máximo 10s).
    - SI el mensaje es un DATO DE TEMPERATURA:
        - Extraer el valor numérico y agregarlo al registro histórico.
        - Si hay menos de 5 muestras: Asignar tendencia "INICIAL".
        - SI hay 5 o más muestras:
            - Calcular el promedio de las últimas 5.
            - Calcular límites superior e inferior (±2%).
            - SI valor actual > límite superior -> Tendencia = "ALTA".
            - SI valor actual < límite inferior -> Tendencia = "BAJA".
            - SINO -> Tendencia = "ESTABLE".
        - Guardar [Hora, Valor, Tendencia] en archivo de texto local.
        - SI el botón no está presionado: Enviar comandos al ESP32 para cambiar el color del LED según la tendencia calculada.

Diagrama de Flujo de la solución Pc Estacion



Código en python

Librerías utilizadas:

1. Librería serial (PySerial)

Descripción general: proporciona el acceso al puerto serie de la computadora. En este proyecto, actúa como puente de comunicación física (vía cable USB)entre el Esp 32 y la PC.

Funciones y métodos utilizados:

- **serial.Serial(puerto, baudios):** Se utiliza para inicializar y abrir el puerto de comunicación especificando la ruta del dispositivo (ej. "/dev/ttyUSB0") y la velocidad en baudios (115200).
- **esp32.in_waiting:** Es una propiedad que se lee dentro del bucle principal para comprobar si hay bytes (mensajes) almacenados en el búfer de entrada esperando a ser leídos. Evita que el programa se quede bloqueado esperando un mensaje que no existe.
- **esp32.readline():** Lee una línea completa de datos proveniente del ESP32 hasta encontrar un salto de línea. Se utiliza para capturar los estados del botón ("BOTON_ON"/"BOTÓN OFF") y las lecturas de temperatura ("24.50c").
- **esp32.write(b'...'):** Se utiliza para mandar los comandos de control de los LEDs (ej. b'E' para encender, b'X' para apagar, b'R' para rojo, etc.) y la orden de lectura de temperatura (b'T').
- **esp32.close():** Libera y cierra el puerto serie de manera segura cuando el programa termina (al presionar Ctrl+C), evitando que el puerto quede "ocupado" en el sistema operativo.

2. Librería time

Descripción general: Proporciona funciones relacionadas con el tiempo del reloj del sistema. Se utiliza para medir el tiempo en el cual está presionado el botón y para generar retardos intencionales (pausas) en la ejecución del código.

Funciones y métodos utilizados:

- **time.time():** Devuelve el tiempo actual expresado en segundos. Se utiliza en la función del botón para capturar el instante exacto en el que se presiona (tiempo_inicio) y restarlo del instante en que se suelta, calculando así el tiempo_total.
- **time.sleep(segundos):** Detiene la ejecución del hilo actual durante una cantidad específica de segundos.
    1. Al inicio del programa (time.sleep(2)) para darle tiempo al ESP32 de reiniciarse y estabilizar la conexión serie.
    2. En el control de luces (time.sleep(0.05)) para mantener el LED encendido durante 50 milisegundos y generar el "destello" visual indicador de nuevo dato.

3. Librería threading

Descripción general: Se utiliza para ejecutar múltiples tareas en segundo plano simultáneamente sin bloquear el bucle principal que debe estar siempre libre escuchando el puerto serie.

Funciones y clases utilizadas:

- **Threading.Thread(target=..., daemon=True)**: Crea un nuevo hilo de ejecución apuntando a una función específica (parpadeo_hilo o ciclo_hilo). El argumento daemon=True asegura que estos hilos en segundo plano se destruyan automáticamente si el programa principal se cierra.
- **hilo.start()**: Da la orden de arranque al hilo creado, poniéndolo a funcionar en paralelo.
- **Threading.Event()**: Crea un objeto de evento que funciona como una "bandera" o interruptor de comunicación entre el hilo principal y los hilos secundarios.
    - **.wait(segundos)**: Se usa dentro de los bucles de los hilos en lugar de time.sleep(). Pausa el hilo por el tiempo indicado (ej.wait(ultimo_tiempo_guardado)), pero con la ventaja de que si recibe una señal, se interrumpe inmediatamente sin esperar a que termine el tiempo.
    - **.set()**: Se utiliza para enviarle la señal al Event. Se usa cuando se suelta el botón o al cerrar el programa para abortar instantáneamente el parpadeo o la espera del reloj.
    - **.clear()**: Limpia la bandera del evento, reseteandolo para permitir que el hilo vuelva a ejecutarse normalmente.

4. Librería datetime

Descripción general: Suministra clases para manipular fechas y horas. En este proyecto, se utiliza exclusivamente para el registro de temperatura.

Funciones y métodos utilizados:

- **datetime.datetime.now()**: Obtiene la fecha y hora local exacta del sistema operativo de la PC en el instante en que se procesa una nueva temperatura.
- **.strftime("%Y-%m-%d %H:%M:%S")**: Formatea el objeto de fecha y hora crudo en una cadena de texto ordenada y legible (Año-Mes-Día Hora:Minuto:Segundo).

Variables Globales y Eventos de Sincronización

Se declaran las variables de estado que serán compartidas a lo largo de todo el ciclo de vida del programa. Al estar definidas al inicio del archivo cualquier hilo o subrutina puede acceder a ellas para leer o modificar su valor en tiempo real, garantizando la comunicación entre el bucle principal y las tareas en segundo plano.

Variables de Estado y Datos:

- `ultimo_tiempo_guardado` (Float): Define el intervalo de tiempo (en segundos) cada cuánto se solicita una nueva lectura de temperatura. Su valor inicial por defecto es de 3.5 segundos, pero es dinámicamente ajustable por el usuario mediante el botón físico.
- `tiempo_inicio` (Float): Almacena la marca de tiempo exacta en la que el usuario presiona el botón. Se utiliza matemáticamente para calcular el tiempo total de pulsación al momento de soltarlo.
- `historial_temperaturas` (List): Funciona como lista vacía inicial que almacena el registro numérico de las lecturas recibidas, necesario para calcular posteriormente el promedio móvil.
- `boton_presionado` (Boolean): Funciona como una bandera (flag) lógica que refleja el estado físico actual del botón. Previene que el sistema solicite temperaturas mientras el usuario interactúa con la placa.
- `programa_detenido` (Boolean): Bandera lógica que indica si el usuario ha pausado intencionalmente la adquisición de datos (activada mediante un clic rápido menor a 1 segundo).

Eventos de Hilos (Threading Events):

- `evento_detener_parpadeo` y `evento_detener_temperatura`: Son objetos instanciados a partir de la clase `threading.Event()`. Operan como interruptores remotos . Permiten que el hilo principal le envíe una señal a los hilos en segundo plano para que se interrumpan y finalicen

```
ultimo_tiempo_guardado = 3.5        # Guarda el último tiempo de lectura asignado (Inicialmente 3.5)
tiempo_inicio = 0                   # Guarda el momento exacto en el que apretás el botón
historial_temperaturas = []         # Lista vacía que almacena las lecturas de temperatura en forma numérica
boton_presionado = False            # Variable que refleja el estado actual del botón (inicialmente en falso)
programa_detenido = False           # Variable que refleja el estado actual del programa

#Eventos de detención de hilos ,Tareas en paralelo
evento_detener_parpadeo = threading.Event()
evento_detener_temperatura = threading.Event()
```

Configuración Inicial del ESP32 (Puerto Serie)

Descripción general: Este bloque establece y habilita la comunicación física entre la computadora y el microcontrolador ESP32 a través de la conexión USB.

Parámetros de configuración: Se definen la ruta del puerto de conexión asignado por el sistema operativo (`"/dev/ttyUSB0"`) y la velocidad de transmisión de datos (`115200` baudios).

Lógica : Utiliza la librería `serial` abrir el puerto de comunicaciones. luego ejecuta una pausa obligatoria de 2 segundos mediante `time.sleep(2)`.

Descripción de las subrutinas:

El programa se encuentra completamente estructurado mediante el uso de subrutinas, donde cada una cumple una función específica para el sistema. Luego, todas estas piezas individuales son llamadas dentro de otra función que cumple el rol de bucle principal continuo, logrando así una estructura modular.

1. Boton_temporizador_Parpadeo(mensaje)

Es la subrutina encargada de la lógica que mide el tiempo exacto de pulsación para determinar si el usuario desea pausar el sistema o configurar el intervalo de muestreo de temperatura.

Parámetros de entrada: mensaje (String): Recibe el estado del botón ("BOTON_ON" al presionar, "BOTON_OFF" al soltar)proveniente del esp32.

Lógica : Al recibir "BOTON_ON", registra la marca de tiempo inicial y lanza el hilo de parpadeo encargado de mostrar el paso de tiempo en segundos. Al recibir "BOTON_OFF", calcula la diferencia de tiempo. Mediante una estructura condicional, asigna un nuevo valor a la variable global ultimo_tiempo_guardado (limitado entre 2.5 y 10s) o activa la bandera programa_detenido si la pulsación fue menor a 1 segundo (clic rápido),luego apaga el hilo encargado del parpadeo.

```
def Boton_temporizador_Parpadeo(mensaje):
    """
    Descripción: Mide cuánto tiempo se mantiene presionado el botón del ESP32 para ajustar
                 el intervalo de muestreo o detener el programa.
    Entradas:
- mensaje (string): "BOTON_ON" (botón presionado) o "BOTON_OFF" (botón soltado).
    Condición de ejecución: Se invoca desde el bucle principal cuando el ESP32 envía
                            una señal de estado del botón.
    """
    global tiempo_inicio, ultimo_tiempo_guardado, boton_presionado, programa_detenido

    if mensaje == "BOTON_ON":
        boton_presionado = True          #Registramos el estado actual del botón
        tiempo_inicio = time.time()      #Guardamos el tiempo actual en el que se presiona
        evento_detener_parpadeo.clear()  #Habilitamos el evento para que el led pueda parpadear
        print("Botón presionado. Contando tiempo y parpadeando...", end="\r")
        parpadeo()                       #Lanzamos el hilo que hace destellar el led

    elif mensaje == "BOTON_OFF":
        boton_presionado = False          #Registramos el estado actual del botón
        tiempo_total = time.time() - tiempo_inicio  # Calcula cuántos segundos fue presionado
        nuevo_tiempo = round(tiempo_total, 2) #Redondeamos el tiempo total a dos decimales

        if nuevo_tiempo < 1.0:             #Tiempo menor a un segundo
            programa_detenido = True       #Indicamos que el programa se debe detener
            print(f"\n---> Click rápido ({nuevo_tiempo}s). Programa DETENIDO.")
```

```
        elif nuevo_tiempo < 2.5:           #Tiempo menor a 2,5 y mayor a un segundo
            programa_detenido = False      #Indicamos que el programa se debe reanudar
            ultimo_tiempo_guardado = 2.5   #Asignamos el tiempo mínimo de 2,5 segundos
            print(f"\n---> Tiempo corto ({nuevo_tiempo}s). Ajustado al MÍNIMO de {ultimo_tiempo_guardado}s")

        elif nuevo_tiempo > 10.0:          #Tiempo entre 2,5 y 10 segundos
            programa_detenido = False      #Indicamos que el programa se debe reanudar
            ultimo_tiempo_guardado = 10.0   #Asignamos el tiempo máximo de 10 segundos
            print(f"\n---> Tiempo excedido ({nuevo_tiempo}s). Ajustado al MÁXIMO de {ultimo_tiempo_guardado}s")

        else:                              #Tiempo entre 2,5 y 10 segundos
            programa_detenido = False      #Indicamos que el programa se debe reanudar
            ultimo_tiempo_guardado = nuevo_tiempo  #Asignamos el tiempo medido
            print(f"\n---> Nuevo muestreo fijado en: {ultimo_tiempo_guardado}s")

        evento_detener_parpadeo.set() # Envía la orden para terminar el hilo de parpadeo
```

2. parpadeo()

Esta función se encarga de hacer parpadear todos los leds durante 50 ms cada 950 ms mientras el botón se encuentra presionado,esto se realiza de manera paralela al bucle principal

Parámetros de entrada: Ninguno.

Lógica : Para no interrumpir la escucha del puerto serie, esta función arranca una subrutina (`parpadeo_hilo()`) que se ejecuta como un hilo en segundo plano . Este hilo envía comandos seriales (encender `b'E'` y apagar `b'X'`) hasta que recibe la señal del evento: `evento_detener_parpadeo`.

```
def parpadeo():
    """
    Descripción: Genera una señal intermitente en los LEDs del ESP32 utilizando un hilo
                 en segundo plano para no bloquear el resto del programa. Indica cuando transcurre un segundo
    Entradas: Ninguna.
    Condición de ejecución: Se inicia exclusivamente cuando la función 'Boton_temporizador_Parpadeo'
                            detecta que el botón ha sido presionado ("BOTON_ON").
    """
    def parpadeo_hilo():
        while not evento_detener_parpadeo.is_set():         # Mientras no se reciba la orden de detener el hilo
            if evento_detener_parpadeo.wait(0.95): break    # Espera casi un segundo, si recibe la orden se cancela
            esp32.write(b'E')                               # Enciende todos los leds
            if evento_detener_parpadeo.wait(0.05): break    # Espera 50 ms
            esp32.write(b'X')                               # Se apagan todos los leds
        esp32.write(b'X')   # Me aseguro que todos los leds estén apagados
    hilo = threading.Thread(target=parpadeo_hilo, daemon=True)  # Prepara esta tarea para correr en segundo plano, se puede detener
    hilo.start()                                            # Arranca el hilo sin frenar el resto del programa
```

3. temporizador_temperatura()

Esta función se encarga de realizar la petición de temperatura al Esp32 cada un determinado tiempo asignado por usuario.

Lógica: Al igual que el parpadeo, lanza un hilo en segundo plano (`ciclo_hilo()`). Este hilo evalúa continuamente las banderas globales (`programa_detenido` y `boton_presionado`). Si el sistema está activo, envía el comando `b'T'` al ESP32 para solicitar una lectura de temperatura, tras lo cual se pausa a sí mismo durante la cantidad de segundos especificada en la variable `ultimo_tiempo_guardado`.

```python
# ========================================
# 3. TEMPORIZADOR DE TEMPERATURA
# ========================================
def temporizador_temperatura():
    """
    Descripción: Hilo en segundo plano que envía la letra 'T' al ESP32 cada cierto
                 intervalo (determinado por 'ultimo_tiempo_guardado') para solicitar una lectura.
    Entradas: Ninguna.
    Condición de ejecución: Se lanza una única vez al iniciar el bucle principal y se ejecuta
                            continuamente, salvo que el programa se detenga manualmente o el botón esté presionado.
    """
    def ciclo_hilo():
        while not evento_detener_temperatura.is_set(): # Mientras no se reciba la orden de apagar el programa
            if evento_detener_temperatura.wait(ultimo_tiempo_guardado): break # Espera los segundos configurados, si recibe señal de apagado, sale
            # Solo solicita temperatura si el programa NO está pausado y si el usuario NO está presionando el botón
            if not programa_detenido and not boton_presionado:
                esp32.write(b'T')                                  # Manda la orden 'T' al ESP32 para que lea el sensor
    hilo_temp = threading.Thread(target=ciclo_hilo, daemon=True)  # Prepara esta tarea para correr en segundo plano, se puede detener
    hilo_temp.start()                                             # Arranca el hilo sin frenar el resto del programa
```

4.procesar_temperatura(valor_str)

Una vez recibida la respuesta a la petición de la temperatura,esta función se encarga de calcular el promedio de las últimos 5 registros de temperatura e indica la tendencia actual de la temperatura

Parámetros de entrada:`valor_str` (String): El valor numérico crudo enviado por el ESP32.

Retorno: Devuelve un String indicando el estado del sistema ("INICIAL", "ESTABLE", "ALTA" o "BAJA").

Lógica : Convierte el dato tipo cadena a punto flotante (`float`) y lo almacena en una lista histórica. Extrae las últimas 5 muestras y calcula su promedio. Luego, establece una banda de histéresis técnica (promedio ± 2%). Si la temperatura actual supera el límite superior, devuelve "ALTA"; si cae por debajo del inferior, "BAJA"; si se mantiene en la banda, "ESTABLE". Si aún no hay 5 muestras recopiladas, devuelve "INICIAL".

```python
def procesar_temperatura(valor_str):
    """
    Descripción: Convierte la lectura de texto a número, la añade al historial y calcula
                 el promedio de las últimas 5 mediciones para determinar la tendencia.
    Entradas:
- valor_str (string): El valor numérico crudo proveniente del ESP32 (ej. "24.50").
    Condición de ejecución: Se llama desde el bucle principal cada vez que se recibe un mensaje válido terminado en la letra 'c'.
    """
    N_EVENTOS = 5                                          # Cantidad de muestras a considerar para calcular el promedio
    X_PORCENTAJE = 2.0                                     # Margen de tolerancia (±2%)

    temp_actual = float(valor_str)                         # Convierte el texto recibido a un número con decimales
    historial_temperaturas.append(temp_actual)             # Agrega la nueva lectura al final de la lista

    ultimos_eventos = historial_temperaturas[-N_EVENTOS:]  # Extrae solamente los últimos 5 datos de la lista

    if len(ultimos_eventos) < N_EVENTOS:                   # Si todavía no tenemos 5 mediciones
        print(f"---> Temp actual: {temp_actual:.2f}°C | Promedio: | Tendencia: INICIAL")  #Devuelve estado INICIAL, sin promedio
        return "INICIAL"

    promedio = sum(ultimos_eventos) / N_EVENTOS # Suma los 5 valores y los divide por 5 para sacar el promedio
    # Calcula la "banda" de normalidad usando el promedio ± 2%
    limite_superior = promedio + (promedio * X_PORCENTAJE / 100.0)
    limite_inferior = promedio - (promedio * X_PORCENTAJE / 100.0)

    if temp_actual > limite_superior:   # Si la temperatura rompe el techo del 2%
            tendencia = "ALTA"
    elif temp_actual < limite_inferior: # Si la temperatura rompe el piso del 2%
            tendencia = "BAJA"
    else:                               # Si se mantiene adentro de los límites
            tendencia = "ESTABLE"

    print(f"---> Temp actual: {temp_actual:.2f}°C | Promedio (N=5): {promedio:.2f}°C | Tendencia: {tendencia}")
    return tendencia                    # Devuelve el resultado final (ALTA, BAJA o ESTABLE)
```

5.guardar_en_archivo(temp_actual, tendencia)

Almacena la lectura actual de la temperatura junto a la tendencia y momento de lectura

Parámetros de entrada:

- `temp_actual` (Float): La temperatura numérica procesada.
- `tendencia` (String): El estado de la tendencia actual.

Lógica: Toma la hora actual (formato `YYYY-MM-DD HH:MM:SS`). Abre (o crea) el archivo `historial.txt` en modo *append* (añadir) para evitar la sobrescritura e inserta una cadena de texto formateada con los parámetros y cierra el archivo automáticamente.

```
def guardar_en_archivo(temp_actual, tendencia):
    """
    Descripción: Escribe una nueva línea en el archivo de texto local 'historial.txt'
                 con la marca de tiempo exacta de la medición.
    Entradas:
- temp_actual (float): La temperatura procesada.
- tendencia (string): El estado ("ALTA", "BAJA", "ESTABLE", "INICIAL").
    Condición de ejecución: Se ejecuta en el bucle principal si la función 'procesar_temperatura'
                            devuelve una tendencia válida.
    """
    # Obtiene la fecha y hora actual del sistema operativo y le da un formato(Año-Mes-Día Hora:Minuto:Segundo)
    ahora = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    # Abre el archivo en modo "a" (añadir) para agregar datos al final sin borrar lo que ya estaba
    with open("historial.txt", "a", encoding="utf-8") as archivo:    #with cierra automáticamente el archivo
        # Arma el texto a guardar usando un f-string y le agrega un salto de línea (\n) al final
        linea = f"Fecha/Hora: {ahora} | Lectura: {temp_actual:.2f}°C | Tendencia: {tendencia}\n"
        archivo.write(linea)   # Inserta esa línea de texto
```

6.bucle_principal() (Explicación)

En esta función se encuentra el bucle principal del programa.Consiste en un bucle infinito en el cual se reciben los datos físicos, se procesan,se llama a otras funciones y activan/desactivan los leds en función de la tendencia actual.

- Parámetros de entrada: Ninguno.
- Lógica operativa:
  1. Arranca el hilo del `temporizador_temperatura()`.
  2. Escucha ininterrumpidamente el búfer de entrada del puerto serie (`esp32.in_waiting`).
  3. Al detectar una cadena, la limpia de caracteres nulos y verificamos el tipo(botón o sensor de temperatura).
  4. Si se trata del estado del botón ,llama a la función Boton_temporizador_Parpadeo(mensaje)
  5. Si es temperatura, invoca a `procesar_temperatura()`, guarda el dato llamando a `guardar_en_archivo()`
  6. Dependiendo de la tendencia actual actualiza el estado de los led de tendencia.
  7. Si se presiona Ctrl+C(`KeyboardInterrupt`)cierra de manera segura los hilos, el puerto serie y el socket de red

```python
def bucle_principal():
    global PC_red
    temporizador_temperatura()   # Lanza el hilo encargado de la medición periódica de temperatura

    try:
        # Bucle infinito
        while True:

            # Verifica si hay algún mensaje esperando en el puerto serie
            if esp32.in_waiting > 0:

                # Lee el mensaje, lo decodifica a texto normal y le quita espacios vacíos a los lados
                mensaje = esp32.readline().decode('utf-8').strip()

                # si el mensaje contiene "BOTON_ON" o "BOTON_OFF"
                if mensaje == "BOTON_ON" or mensaje == "BOTON_OFF":
                    Boton_temporizador_Parpadeo(mensaje)  #Se llama a la función

                # El mensaje termina en 'c' lo que indica que es un valor de temperatura
                elif mensaje.endswith("c"):
                    solo_numero = mensaje[:-1]            # Eliminamos el carácter 'c'
                    tendencia = procesar_temperatura(solo_numero)  #Enviamos el texto a la función y almacenamos el retorno

                    if tendencia is not None:              # Si la tendencia es no nula
                        temp_num = float(solo_numero)      # Transformamos la cadena de texto a numérico tipo float

                        # Guardado local
                        guardar_en_archivo(temp_num, tendencia) # Guardamos los datos

                        # Envío por red
                        if PC_red is not None:
                            try:
                                datos_red = f"{temp_num},{tendencia}\n"     # Prepara el paquete de datos
                                PC_red.send(datos_red.encode('utf-8'))      # Lo envía por el socket
                            except:
                                print("\n[RED] Error: Se perdió la conexión con la PC.")
                                PC_red = None                               # Desactiva la red si falla

                        # Control de Luces
                        if tendencia is not None and not boton_presionado:  # Si la tendencia no es nula y el botón no está presionado

                            # Destello rápido indicador de nuevo dato
                            esp32.write(b'E')                               # Enciende todos los leds
                            time.sleep(0.05)
                            esp32.write(b'X')                               # Apaga todos los leds

                            if tendencia == "INICIAL":
                                esp32.write(b'E')                           # Enciende todos los leds
                            elif tendencia == "ESTABLE":
                                esp32.write(b'V')                           # Enciende el led verde
                            elif tendencia == "ALTA":
                                esp32.write(b'R')                           # Enciende el led rojo
                            elif tendencia == "BAJA":
                                esp32.write(b'A')                           # Enciende el led azul

    except KeyboardInterrupt:                                               # Ctrl+C para cerrar programa
        print("\nCerrando conexión...")
        evento_detener_parpadeo.set()                                       # Apaga el hilo de parpadeo
        evento_detener_temperatura.set()                                    # Apaga el hilo de medición
        esp32.close()                                                       # Cierra el puerto USB
        if PC_red: PC_red.close()                                           # Cierra la conexión de red (el socket)
```

Explicación del funcionamiento del código con capturas de pantalla

Al comenzar el programa, el algoritmo realiza peticiones de temperatura periódicamente, respetando el tiempo inicial asignado,en cada medición se realiza un parpadeo. Durante las primeras 4 mediciones, la tendencia se indica cómo "inicial". Una vez alcanzadas las 5 lecturas, se comienza a mostrar la tendencia actual y se enciende el LED correspondiente.

```
---> Temp actual: 23.78°C | Promedio: | Tendencia: INICIAL
---> Temp actual: 23.76°C | Promedio: | Tendencia: INICIAL
---> Temp actual: 23.74°C | Promedio: | Tendencia: INICIAL
---> Temp actual: 23.72°C | Promedio: | Tendencia: INICIAL
---> Temp actual: 23.71°C | Promedio (N=5): 23.74°C | Tendencia: ESTABLE
---> Temp actual: 23.70°C | Promedio (N=5): 23.73°C | Tendencia: ESTABLE
---> Temp actual: 25.24°C | Promedio (N=5): 24.02°C | Tendencia: ALTA
---> Temp actual: 25.71°C | Promedio (N=5): 24.42°C | Tendencia: ALTA
---> Temp actual: 25.98°C | Promedio (N=5): 24.87°C | Tendencia: ALTA
---> Temp actual: 26.14°C | Promedio (N=5): 25.35°C | Tendencia: ALTA
---> Temp actual: 26.29°C | Promedio (N=5): 25.87°C | Tendencia: ESTABLE
---> Temp actual: 26.38°C | Promedio (N=5): 26.10°C | Tendencia: ESTABLE
---> Temp actual: 25.33°C | Promedio (N=5): 26.02°C | Tendencia: BAJA
---> Temp actual: 24.97°C | Promedio (N=5): 25.82°C | Tendencia: BAJA
---> Temp actual: 24.84°C | Promedio (N=5): 25.56°C | Tendencia: BAJA
---> Temp actual: 24.73°C | Promedio (N=5): 25.25°C | Tendencia: BAJA
---> Temp actual: 24.65°C | Promedio (N=5): 24.90°C | Tendencia: ESTABLE
---> Temp actual: 24.60°C | Promedio (N=5): 24.76°C | Tendencia: ESTABLE
```

Paralelamente, se almacena en un archivo de texto la lectura de temperatura, la tendencia y la hora exacta en la que se realizó la medición.

```
2026-11-08 18:50:19,23.78,INICIAL
2026-11-08 18:50:22,23.76,INICIAL
2026-11-08 18:50:26,23.74,INICIAL
2026-11-08 18:50:29,23.72,INICIAL
2026-11-08 18:50:33,23.71,ESTABLE
2026-11-08 18:50:36,23.7,ESTABLE
2026-11-08 18:50:40,25.24,ALTA
2026-11-08 18:50:43,25.71,ALTA
2026-11-08 18:50:47,25.98,ALTA
2026-11-08 18:50:50,26.14,ALTA
2026-11-08 18:50:54,26.29,ESTABLE
2026-11-08 18:50:57,26.38,ESTABLE
2026-11-08 18:51:01,25.33,BAJA
2026-11-08 18:51:04,24.97,BAJA
2026-11-08 18:51:08,24.84,BAJA
2026-11-08 18:51:11,24.73,BAJA
2026-11-08 18:51:15,24.65,ESTABLE
2026-11-08 18:51:18,24.6,ESTABLE
```

Al presionar el botón, los LEDs comienzan a parpadear cada segundo para indicar el paso del tiempo. Si el botón se presiona durante menos de un segundo, el programa se detiene. Si se presiona por más de un segundo pero menos de 2,5 segundos, se asigna el tiempo de muestreo mínimo de 2,5 segundos. Si se mantiene presionado durante más de 10 segundos, se le asigna el tiempo máximo de 10. Finalmente, al soltar el botón, se reanudan las peticiones de temperatura con el nuevo tiempo de muestreo configurado.

```
Botón presionado. Contando tiempo y parpadeando...
---> Click rápido (0.21s). Programa DETENIDO.
Botón presionado. Contando tiempo y parpadeando...
---> Tiempo corto (1.37s). Ajustado al MÍNIMO de 2.5s
---> Temp actual: 24.09°C | Promedio (N=5): 24.00°C | Tendencia: ESTABLE
---> Temp actual: 24.10°C | Promedio (N=5): 24.03°C | Tendencia: ESTABLE
Botón presionado. Contando tiempo y parpadeando...
---> Nuevo muestreo fijado en: 3.23s
---> Temp actual: 24.16°C | Promedio (N=5): 24.07°C | Tendencia: ESTABLE
---> Temp actual: 24.18°C | Promedio (N=5): 24.11°C | Tendencia: ESTABLE
Botón presionado. Contando tiempo y parpadeando...
---> Tiempo excedido (12.19s). Ajustado al MÁXIMO de 10.0s
---> Temp actual: 24.31°C | Promedio (N=5): 24.17°C | Tendencia: ESTABLE
```

Parte B

Para procesar los datos que entrega la estación de monitoreo se requiere hacer un programa que realice un análisis estadístico de las lecturas realizadas. El programa toma el archivo generado por el sistema y realiza la siguientes operaciones:

- Imprime la cantidad de lecturas realizadas por la estación.
- Imprime la menor y la mayor temperatura detectada, junto a la fecha y hora de la muestra.
- Calcula e imprime la media, mediana, moda y alguna medida de dispersión de los datos (ej. desviación estándar o varianza). Justifique qué técnica de dispersión se adapta mejor a los datos producidos.

Sobre la resolución:

- El ejercicio debe ser codificado en C.
- Para su resolución diseñe y utilice una estructura que modele los datos de forma estructurada.
- Para la gestión de los datos utilice memoria dinámica.
- Esto será realizado en la misma Raspberry anterior o en Pc en el caso de usar arduino o simulador

Introducción

Para el cumplimiento de los requisitos de este inciso, se elaboró un programa en C que será ejecutado por la PC estación, la cual se encarga de la comunicación con el ESP32 y del envío de datos a la PC principal o servidor. El programa recibe un archivo que contiene las lecturas de temperatura, tendencia y la fecha/hora en la que se realizó cada medición. Con esta información, el código opera estadísticamente para encontrar datos relevantes como la media, mediana, moda y, en este caso, la desviación estándar.

¿Por qué utilizar la desviación estándar?

Se optó por la desviación estándar porque es mucho más fácil de interpretar. El problema de usar la varianza es que su fórmula matemática eleva los números al cuadrado, dando como resultado "Grados Centígrados al cuadrado" (°C²), una medida que en la vida real no tiene sentido para hablar de temperatura. Al aplicarle la raíz cuadrada a ese cálculo obtenemos la desviación estándar, lo que nos devuelve a la unidad original (°C) y nos permite interpretar fácilmente la lectura. Es mucho más claro y útil decir que "la temperatura promedio fue de 25 °C, con variaciones de ± 1.5 °C" que intentar analizar datos térmicos al cuadrado.

Requerimientos y Restricciones

Requerimientos:

- Debe ser capaz de abrir y leer de forma secuencial un archivo de texto (`historial.txt`) generado a partir de las mediciones del ESP32.
- Debe extraer e interpretar correctamente tres tipos de datos: Fecha/Hora (texto), Temperatura (valor numérico decimal) y Tendencia (texto).

- Debe calcular de manera precisa los valores de Temperatura Mínima, Máxima, Media (promedio), Mediana, Moda y Desviación Estándar.
- Debe imprimir en la pantalla de la terminal un reporte estadístico claro, formateado y de fácil lectura para el usuario.

Restricciones :

- Uso de Memoria Dinámica
- Es necesario que el programa devuelva al sistema operativo toda la memoria solicitada mediante la función `free()` antes de finalizar su ejecución.

Algoritmo General de Alto Nivel de Abstracción

- Intentar abrir el archivo de registro (`historial.txt`). Si no existe o hay un error de lectura, abortar la ejecución.
- Solicitar al sistema operativo un bloque inicial de memoria RAM dinámica

Bucle de Lectura:

- Leer el archivo de texto renglón por renglón hasta llegar al final.
- Si la cantidad de renglones supera el límite de la memoria reservada, solicitar automáticamente una expansión del bloque de RAM, sin perder los datos previos.
- se extrae la fecha, la temperatura y la tendencia, y guardarlas ordenadamente en la memoria RAM.
- Al finalizar la lectura total, cerrar el archivo físico.

Análisis Estadístico

- se recorre todos los datos almacenados en la memoria.
- se suma el total de las temperaturas para calcular la Media (Promedio).
- Identificar los valores extremos (Mínimo y Máximo) y registrar en qué posición de memoria ocurrieron para no perder la fecha exacta del suceso.
- Recorrer nuevamente los datos para comparar cada lectura individual contra el promedio general, aplicando la fórmula matemática para obtener la Desviación Estándar.
- Extraer únicamente los valores de temperatura y se aplica un algoritmo de ordenamiento para organizar todas las temperaturas estrictamente de menor a mayor.
- Mediana: se identifica y extrae el valor numérico que quedó ubicado exactamente en el centro físico de la lista ordenada.
- Moda: se recorre la lista ordenada contando las repeticiones consecutivas de cada valor para determinar cuál fue la temperatura dominante (la de mayor racha).

Presentación de Resultados:

- Imprimir en pantalla un reporte final formateado, mostrando la cantidad total de muestras procesadas y todos los indicadores estadísticos calculados.
- Se libera la memoria utilizada.

Librerías utilizadas

1. Librería <stdio.h> (Standard Input/Output)

Se encarga de manejar toda la entrada y salida de información, tanto en la pantalla (terminal) como en la lectura de archivos del disco duro.

- fopen(): Abre el archivo historial.txt en modo lectura.
- fgets(): Lee el archivo de texto renglón por renglón de manera segura, evitando que el programa lea más allá del límite de cada línea.
- sscanf(): Es un extractor de texto. Agarra el renglón que leyó fgets, busca las comas, y recorta la fecha, la temperatura y la tendencia para guardarlas en variables separadas.
- printf(): Imprime los textos y los resultados estadísticos finales en la pantalla de la computadora.
- fclose(): Cierra el archivo de texto de forma segura una vez que se terminó de leer, liberando el recurso.

2. Librería <stdlib.h> (Standard Library)

Proporciona utilidades fundamentales del sistema operativo. En este proyecto, es la responsable absoluta de cumplir con la consigna de memoria dinámica y ordenamiento.

- malloc(): *(Memory Allocation)* Le pide al sistema operativo que reserve un bloque inicial de memoria RAM (arrancamos pidiendo espacio para 10 registros).
- realloc(): *(Re-Allocation)* Expande el tamaño de la memoria RAM reservada si el archivo resulta ser más grande de lo esperado, conservando los datos anteriores intactos.
- free(): Libera toda la memoria RAM solicitada y se la devuelve a la computadora justo antes de que el programa termine (evita colapsos de memoria).
- qsort(): *(Quick Sort)* Ejecuta un algoritmo de ordenamiento ultrarrápido para acomodar todas las temperaturas de menor a mayor (requisito para la Mediana y la Moda).

3. Librería <math.h> (Math)

Contiene operaciones matemáticas avanzadas que C no trae por defecto. (Requiere compilar con -lm).

- pow(): *(Power)* Eleva un número a una potencia. Lo usamos en la fórmula de la desviación estándar para elevar al cuadrado la diferencia entre cada lectura y el promedio.
- sqrt(): *(Square Root)* Calcula la raíz cuadrada. Se aplica al final del cálculo de dispersión para obtener la Desviación Estándar expresada en la unidad original (°C).

4. Librería <string.h> (String)

Proporciona las herramientas exclusivas para manipular, copiar y comparar las cadenas de texto (arreglos de caracteres) que almacenamos en la memoria dinámica.

Subrutinas creadas

comparar_temperaturas()

Esta función indica al algoritmo de ordenamiento `qsort` cómo debe comparar números decimales, ya que por defecto la computadora no sabe cómo ordenarlos.

- Parámetros de entrada (`const void *a, const void *b`): Recibe dos punteros "genéricos".
- Conversión (Casting): Mediante la instrucción `float t1 = *(const float *)a;`, la subrutina fuerza al sistema a interpretar esos datos como números con coma flotante (`float`), permitiendo así hacer cálculos matemáticos.
- Lógica: Una vez convertidos, comparar matemáticamente ambos números y le devuelve:
    - Retorna `-1`: Si el primer valor es menor (lo coloca a la izquierda).
    - Retorna `1`: Si el primer valor es mayor (lo coloca a la derecha).
    - Retorna `0`: Si ambos valores son idénticos (mantiene posición actual).

```c
// Función auxiliar para ordenar las temperaturas (necesario para Mediana y Moda)
// Necesaria para usar la función qsort de C y calcular la Mediana y la Moda
int comparar_temperaturas(const void *a, const void *b) {
    // Convierte los datos de memoria a formato decimal (float)
    float t1 = *(const float *)a;
    float t2 = *(const float *)b;
    // Compara los valores para ordenarlos de menor a mayor
    if (t1 < t2) return -1; // t1 es menor, va primero
    if (t1 > t2) return 1;  // t1 es mayor, va después
    return 0; // Si son iguales, se mantienen en su lugar
}
```

Código C

```c
#include <stdio.h>
#include <stdlib.h> // Proporciona las funciones para memoria dinámica (malloc, realloc, free, qsort)
#include <string.h> // Permite la manipulación de cadenas de texto
#include <math.h> // Proporciona funciones matemáticas (pow, sqrt) para la desviación estándar

// 1. Estructura de los datos de cada muestra
// Se utiliza typedef para definir un nuevo tipo de dato llamado "Registro"
typedef struct {
    char fecha_hora[64];    //Reservamos un espacio en memoria para 64 caracteres
    float temperatura;      //Almacena el valor numérico decimal de la lectura
    char tendencia[32];     //Reservamos un espacio en memoria para 32 caracteres
} Registro;

// ========================================
// Comaprar_temperaturas
// ========================================

// Función auxiliar para ordenar las temperaturas (necesario para Mediana y Moda)
// Necesaria para usar la función qsort de C y calcular la Mediana y la Moda
int comparar_temperaturas(const void *a, const void *b) {
    // Convierte los datos de memoria a formato decimal (float)
    float t1 = *(const float *)a;
    float t2 = *(const float *)b;
    // Compara los valores para ordenarlos de menor a mayor
    if (t1 < t2) return -1; // t1 es menor, va primero
    if (t1 > t2) return 1;  // t1 es mayor, va después
    return 0; // Si son iguales, se mantienen en su lugar
}
```

```c
int main() {

// ========================================
// Lectura del archivo y Memoria dinamica
// ========================================
    // Abrimos el archivo "historial.txt" en modo lectura ("r" de read)
    FILE *archivo = fopen("historial.txt", "r");
    // Si el archivo no existe, avisamos y detenemos el programa
    if (archivo == NULL) {
        printf("Error: No se pudo abrir el archivo historial.txt\n");
        return 1;
    }

    // 2. Gestión de memoria dinámica
    int capacidad = 10; // Empezamos reservando espacio para 10 lecturas
    int cantidad = 0; // Contador para saber cuántas lecturas vamos guardando
    // Reservamos la memoria inicial multiplicando la capacidad (10) por el tamaño de un "Registro"
    Registro *datos = (Registro *)malloc(capacidad * sizeof(Registro));

    char linea[128]; // Espacio temporal para guardar cada renglón del texto

    // Leemos el archivo línea por línea hasta que se termine
    while (fgets(linea, sizeof(linea), archivo)) {

        // Si nos quedamos sin espacio (ej: llegamos a la lectura 11)...
        if (cantidad >= capacidad) {
            capacidad *= 2;  // Duplicamos la capacidad (de 10 pasa a 20, 40, 80...)
            // Expandimos la memoria dinámicamente sin perder los datos anteriores
            datos = (Registro *)realloc(datos, capacidad * sizeof(Registro));
        }

        // Extraer datos asumiendo el formato: "Fecha Hora,Temperatura,Tendencia"
        // Ejemplo: "07/11/2026 14:30:00,24.5,ALTA"
        //%[^,] guarda la fecha, %f guarda el decimal, y %s guarda la tendencia
        sscanf(linea, "%[^,],%f,%s", datos[cantidad].fecha_hora, &datos[cantidad].temperatura, datos[cantidad].tendencia);
        cantidad++; // Sumamos 1 al contador de lecturas
    }
    fclose(archivo);  //Cerramos el archivo
```

```c
// ========================================
// Mínimo, Máximo y Media
// ========================================

    // 3. Variables para las estadísticas
    float suma = 0;                    // Variable para sumar todas las temperaturas
    int indice_min = 0, indice_max = 0; // Guarda la posición donde está la temperatura más baja y alta

    // Creamos un arreglo dinámico temporañ solo para las temperaturas.
    float *solo_temps = (float *)malloc(cantidad * sizeof(float)); // Arreglo para facilitar ordenamiento

    // Buscar Min, Max y sumar para la Media

    // Recorremos todos los registros uno por uno
    for (int i = 0; i < cantidad; i++) {
        solo_temps[i] = datos[i].temperatura;   //Tomamos solo las temperaturas
        suma += datos[i].temperatura;           // Vamos sumando el valor para el promedio

        // Comparamos para encontrar los extremos
        if (datos[i].temperatura < datos[indice_min].temperatura)
        indice_min = i; // Si encontramos uno menor, actualizamos la posición mínima
        if (datos[i].temperatura > datos[indice_max].temperatura)
        indice_max = i; // Si encontramos uno mayor, actualizamos la posición máxima
    }


    // Calculamos la Media (Promedio)
    //Es la suma de todas las temperaturas dividida por la cantidad total de lecturas.
    float media = suma / cantidad;

// ========================================
// Desvacion Estandar
// ========================================
    // Calcular Desviación Estándar (Dispersión)
    //Medida de dispersión que indica la variación promedio de un conjunto de datos respecto a su media aritmética

    float suma_diferencias = 0;
    for (int i = 0; i < cantidad; i++) {
        // Resta la temperatura actual menos el promedio, y lo eleva al cuadrado (pow)
        suma_diferencias += pow(datos[i].temperatura - media, 2);
    }
    // Divide por la cantidad y le saca la raíz cuadrada (sqrt)
    float desviacion_estandar = sqrt(suma_diferencias / cantidad);
```

```c
// ========================================
// Mediana y Moda
// ========================================

    // Ordenar arreglo de temperaturas para Mediana y Moda
    //Utilizamso qsort y le pasamos la  función comparar_temperaturas
    qsort(solo_temps, cantidad, sizeof(float), comparar_temperaturas);

    // Mediana
    //Medida de tendencia central que representa el valor que divide la muestra ordenada en dos mitades simétricas
    float mediana;
    if (cantidad % 2 == 0) {  // Si la cantidad de lecturas es un número par, no hay un solo centro, hay dos.
        // Agarramos los dos números del centro y los promediamos
        mediana = (solo_temps[cantidad / 2 - 1] + solo_temps[cantidad / 2]) / 2.0;
    } else {
        // Si es impar, simplemente agarramos el número que quedó exactamente en el medio
        mediana = solo_temps[cantidad / 2];
    }

    // Moda
    //Medida de tendencia central que identifica el valor con la mayor frecuencia en la distribución de datos
    float moda = solo_temps[0]; // Empezamos asumiendo que el primero es la moda
    int max_repeticiones = 1; // Guardará la "racha" más larga de repeticiones encontradas
    int conteo_actual = 1;

    // Recorremos las temperaturas ya ordenadas
    for (int i = 1; i < cantidad; i++) {  // Recorremos las temperaturas ya ordenadas
        // Si el número actual es igual al anterior, el contador crece
        if (solo_temps[i] == solo_temps[i - 1]) {
            conteo_actual++;
        } else {// Si el número el numero cambio vuelve a 1
            conteo_actual = 1;
        }
        // Si la racha que acabamos de contar le gana a la mayor que teníamos guardada...
        if (conteo_actual > max_repeticiones) {
            max_repeticiones = conteo_actual;   // Guardamos el nuevo récord de repeticiones
            moda = solo_temps[i];               // Y declaramos a esta temperatura como la nueva moda
        }
    }
```

```c
// ========================================
// Impresión de Resultados y Limpieza
// ========================================
   // %d números enteros
   //%.2f muestra el número decimal limitado a 2 decimales
   // %s muestra el texto (string)
    printf("--- REPORTE ESTADÍSTICO DE TEMPERATURA ---\n");
    printf("Cantidad total de lecturas : %d\n", cantidad);
    printf("Temp. Mínima detectada     : %.2f °C (Fecha/Hora: %s)\n", datos[indice_min].temperatura, datos[indice_min].fecha_hora);
    printf("Temp. Máxima detectada     : %.2f °C (Fecha/Hora: %s)\n", datos[indice_max].temperatura, datos[indice_max].fecha_hora);
    printf("Media (Promedio)           : %.2f °C\n", media);
    printf("Mediana                    : %.2f °C\n", mediana);
    printf("Moda                       : %.2f °C\n", moda);
    printf("Desviación Estándar        : %.2f °C\n", desviacion_estandar);

    // Liberar la memoria dinámica utilizada
    free(solo_temps);
    free(datos);

    return 0;
}
```

Captura de pantalla demostrando el funcionamiento

Una vez tenemos nuestro registro de temperatura proporcionado por el código de python debemos ejecutar nuestro código C,el cual previamente fue compilado mediante el comando **gcc codigo.c -o codigo -lm**

```
2026-11-08 18:50:19,23.78,INICIAL
2026-11-08 18:50:22,23.76,INICIAL
2026-11-08 18:50:26,23.74,INICIAL
2026-11-08 18:50:29,23.72,INICIAL
2026-11-08 18:50:33,23.71,ESTABLE
2026-11-08 18:50:36,23.7,ESTABLE
2026-11-08 18:50:40,25.24,ALTA
2026-11-08 18:50:43,25.71,ALTA
2026-11-08 18:50:47,25.98,ALTA
2026-11-08 18:50:50,26.14,ALTA
2026-11-08 18:50:54,26.29,ESTABLE
2026-11-08 18:50:57,26.38,ESTABLE
2026-11-08 18:51:01,25.33,BAJA
2026-11-08 18:51:04,24.97,BAJA
2026-11-08 18:51:08,24.84,BAJA
2026-11-08 18:51:11,24.73,BAJA
2026-11-08 18:51:15,24.65,ESTABLE
2026-11-08 18:51:18,24.6,ESTABLE
```

Para ejecutarlo simplemente escribimos en la consola el comando ./código y el programa realizará el análisis estadístico devolviendonos el resumen por la propia terminal

```
--- REPORTE ESTADÍSTICO DE TEMPERATURA ---
Cantidad total de lecturas : 18
Temp. Mínima detectada     : 23.70 °C (Fecha/Hora: 2026-11-08 18:50:36)
Temp. Máxima detectada     : 26.38 °C (Fecha/Hora: 2026-11-08 18:50:57)
Media (Promedio)           : 24.85 °C
Mediana                    : 24.78 °C
Moda                       : 23.70 °C
Desviación Estándar        : 0.94 °C
```

Parte C

Para analizar los datos que entrega la estación de monitoreo se requiere hacer un programa que realice una visualización gráfica de las lecturas realizadas. El programa consta de 2 partes:

Si se utiliza Raspberry Pi, la comunicación con la PC se realizará mediante sockets TCP/IP. Si se utiliza PC junto a Arduino/ESP32 o el simulador, la comunicación se establecerá entre dos computadoras.

- Un programa en Raspberry/PC principal en el cual se deberá configurar un socket TCP/IP y que, una vez establecido este, proceda a enviar las lecturas de los sensores por el mismo.
- Un programa en una computadora extra que se conectará al socket previamente establecido en la Raspberry PI o PC principal.
- Proceder a la visualización de los datos en la computadora final.
- Estos serán presentados de una manera gráfica (se sugiere la librería matplotlib) y deberán actualizarse a medida que se reciban los datos obtenidos en el servidor, es decir, en tiempo real.

Sobre la resolución:

- El ejercicio debe ser codificado en Python.

Introducción

Para la última etapa del proyecto, se desarrolló un código en Python que permite visualizar mediante un gráfico la medición de temperatura actual en una computadora remota, conectándose a través de un socket TCP/IP.

Requerimientos y restricciones específicas

1. Requerimientos

- Es necesario disponer de una red local para poder enlazar ambos dispositivos mediante la comunicación por socket TCP.
- Para representar visualmente los datos obtenidos durante la comunicación es necesario la librería externa `matplotlib`
- Es necesario envíar los datos en formato de texto (codificados en UTF-8), separados exactamente por una coma y sin caracteres basura. `Temperatura,Tendencia`.

2. Restricciones

- Para evitar el consumo excesivo y desbordamiento de memoria RAM, el gráfico está restringido a almacenar y mostrar únicamente las últimas 30 muestras recibidas.

- El eje Y del gráfico no es libre; está restringido a ajustarse automaticamente
- Si la PC externa se desconecta o no está disponible, el sistema debe continuar operando de forma local (Codigo pc esp32)

Librerias adicionales utilizadas

Librería `socket`

Descripción general: Proporciona acceso a la interfaz de red de bajo nivel del sistema operativo. Implementa la arquitectura de red TCP/IP (Cliente-Servidor) permitiendo que el script local transmita los datos en tiempo real hacia una computadora remota dentro de la misma red local.

Funciones y métodos utilizados en la PC Estación

- **socket.socket(socket.AF_INET, socket.SOCK_STREAM):** Inicializa el socket definiendo la familia de direcciones `AF_INET` (para usar direcciones IPv4) y el tipo `SOCK_STREAM` (para utilizar el protocolo TCP, garantizando que los paquetes de datos lleguen completos y en orden sin pérdida de información).
- **PC_red.connect((IP, PUERTO)):** Ejecuta la petición de conexión hacia la IP de la máquina principal en el puerto especificado (5000).
- **PC_red.send(datos.encode('utf-8')):** Toma la cadena de texto con la temperatura y la tendencia, la codifica en un formato de bytes estándar (utf-8) y la inyecta en la red hacia el servidor.
- **PC_red.close():** Cierra ordenadamente la conexión de red al detener el programa.

Funciones y métodos utilizados en la PC Principal:

- **socket.socket(socket.AF_INET, socket.SOCK_STREAM):** Inicializa el objeto de red local preparándolo para operar bajo el esquema IPv4 y protocolo TCP.
- **servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1):** Modifica las opciones del socket a nivel del sistema operativo para forzar la liberación y reutilización del puerto instantáneamente al cerrar el programa, evitando el bloqueo temporal (error de puerto en uso).
- **servidor.bind((HOST, PORT)):** Vincula o enlaza el socket creado a una dirección IP y a un puerto físico específico, habilitando a la máquina para recibir el tráfico entrante.
- **servidor.listen(1):** Pone al socket en estado de escucha activa, configurando un límite máximo de un (1) cliente conectado en simultáneo.
- **servidor.accept():** Bloquea la ejecución del hilo esperando una conexión entrante. Cuando el equipo estación se conecta, retorna un nuevo objeto de conexión y la dirección IP del emisor.
- **conn.recv(1024).decode('utf-8'):** Extrae hasta 1024 bytes del búfer de red entrante y los decodifica de binario a texto legible para su posterior separación y graficación.

Configuración de Red (Sockets TCP/IP)

1. Entorno Cliente (PC Estación)

- **Parámetros de configuración:** Se define estáticamente la dirección IP de la PC de destino (`IP_PC_PRINCIPAL = 'xxx.xxx.x.xxx'`) y el puerto de escucha (`PORT = 5000`).
- **Lógica operativa:** La creación y conexión de la interfaz de red se realiza dentro de una estructura de control de excepciones (`try/except`). El proceso consta de dos pasos: la creación del socket TCP/IPv4 y la invocación del método `.connect()`. Si la conexión es exitosa, el programa queda listo para emitir datos. Si el servidor destino está apagado, el bloque `except` captura el error, notifica por consola y asigna un valor nulo (None) a la variable de red. Gracias a esto, el bucle principal sabe que no hay red disponible y continúa trabajando en modo exclusivamente local (tolerancia a fallos).

```
IP_PC_PRINCIPAL = '192.168.1.112'       #Ip PC principal
PORT = 5000                             # Puerto de escucha

print("Intentando conectar...")
try:
    # Prepara la "tubería" (socket) usando el protocolo de internet (IPv4) y TCP
    PC_red = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    PC_red.connect((IP_PC_PRINCIPAL, PORT))                    # Intenta conectarse
    print("[RED] ¡Conectado al PC principal!\n")
except:      # En caso de no poder realizar la conexión
    print("[RED] No se encontró la PC. El programa funcionará localmente.\n")
    PC_red = None                               # Variable nula para que el bucle principal sepa que no hay red
```

2. Entorno Servidor (PC Principal de Monitoreo)

- **Parámetros de configuración:** Se define el alojamiento genérico (`HOST = '0.0.0.0'`) para indicar que el servidor escuchará peticiones desde cualquier interfaz de red disponible (no solo localhost) en el puerto asignado (`PORT = 5000`).
- **Lógica operativa:** A diferencia de la estación recolectora, el servidor debe gestionar un estado de escucha permanente. Se configura el socket aplicando el parámetro SO_REUSEADDR para evitar bloqueos del sistema operativo, se vincula al puerto mediante `.bind()` y se activa la escucha con `.listen()`.
- **Concurrencia:** Todo este proceso de red se encapsula dentro de una función independiente y se ejecuta en un hilo en segundo plano (`threading.Thread`). Esto es crucial para que el bloqueo del método `.accept()` (que se queda esperando a la estación) no congele la interfaz gráfica generada por Matplotlib en el hilo principal. Una vez conectado, el hilo entra en un bucle infinito de lectura mediante `.recv()`, inyectando constantemente los datos en los búferes globales para su graficación.

```
# HOST = '0.0.0.0' indica que escuchará conexiones desde cualquier IP en la red local
HOST = '0.0.0.0'
PORT = 5000      # Puerto de comunicación
```

```
# Inicialización del socket TCP/IPv4
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# SO_REUSEADDR libera el puerto inmediatamente al cerrar el programa para evitar conflictos y reinterntar la conexion
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind((HOST, PORT)) #toma el objeto socket vacío y lo asigna a una ip y a un puerto especifico
servidor.listen(1) # El servidor se pone a la escucha (1 cliente máximo a la vez)
```

Algoritmo General de Alto Nivel de Abstracción

El siguiente algoritmo describe la lógica del flujo principal del programa de forma general

Al iniciar el programa:

- Inicializar las variables globales de almacenamiento (listas de tiempo y temperatura).
- Iniciar un hilo exclusivo para la comunicación de red.
- Configurar la ventana, los ejes del gráfico y la tasa de refresco de la animación.

Hilo de Red (Mientras el programa esté en ejecución):

- Configurar y abrir el puerto de escucha (Socket TCP, Puerto 5000).
- Esperar continuamente hasta que la PC Estación se conecte.
- MIENTRAS la PC Estación mantenga la conexión activa:
    - Leer el búfer de red entrante.
    - SI el paquete de red está vacío:
        - Registrar desconexión, cerrar el enlace y volver a esperar una nueva conexión.
    - SI el paquete contiene datos válidos (formato "Temperatura,Tendencia"):
        - Extraer el valor numérico de la temperatura.
        - Agregar el valor de temperatura y el instante de tiempo a las listas globales.
        - SI la cantidad de datos en la lista es mayor a 30 (Ventana móvil):
            - Eliminar el dato más antiguo para evitar el consumo infinito de memoria.

Hilo Graficador (Se ejecuta automáticamente cada 500 milisegundos):

- Limpiar el lienzo para borrar la traza gráfica anterior.
- Dibujar la nueva curva uniendo los puntos de las listas globales de tiempo y temperatura.
- SI existen datos actualmente en las listas:
    - Calcular el valor de temperatura máximo y mínimo del conjunto actual.
    - Establecer el límite superior del eje Y (Máximo + 2.0 °C).
    - Establecer el límite inferior del eje Y (Mínimo - 2.0 °C).
    - Ajustar la vista del gráfico a estos nuevos límites (Auto-escalado dinámico).
- Configurar textos, títulos y cuadrícula de fondo.
- Mostrar la interfaz en pantalla.

Código Python Pc principal

```python
import socket
import threading
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ========================================
# CONFIGURACIÓN DE RED (SERVIDOR TCP)
# ========================================
# HOST = '0.0.0.0' indica que escuchará conexiones desde cualquier IP en la red local
HOST = '0.0.0.0'
PORT = 5000      # Puerto de comunicación TCP asignado

# Variables globales
tiempos_x = []       # Almacena el eje X
temperaturas_y = []  # Almacena el eje Y (valores de temperatura)
contador_tiempo = 0  # Índice que aumenta con cada paquete recibido

# ========================================
# HILO RECEPTOR DE DATOS
# ========================================
def recibir_datos():
    global contador_tiempo

    # Inicialización del objeto socket TCP/IPv4
    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # SO_REUSEADDR libera el puerto inmediatamente al cerrar el programa para evitar conflictos y permitir reconexiones rápidas
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    # Asigna (bind) el objeto socket a la IP y puerto definidos
    servidor.bind((HOST, PORT))

    # El servidor se pone a la escucha (1 cliente máximo en la sala de espera)
    servidor.listen(1)

    print(f"--- SERVIDOR GRÁFICO INICIADO ---")
    print(f"Esperando conexion de PC en el puerto {PORT}...")
```

```python
while True: # Bucle infinito
    try:
        # .accept() el hilo se pausa aquí hasta que un cliente se conecte
        # Devuelve 'conn' (el canal de comunicación) y 'addr' (la IP y puerto del cliente)
        conn, addr = servidor.accept()
        ip_PCesp = addr[0]

        print(f"\n[RED] ¡Computadora conectada desde {ip_PCesp}!\n")

        while True: # Bucle infinito Ciclo de lectura continua de datos del cliente conectado

            # Recibe hasta 1024 bytes del canal y los decodifica de sistema binario a texto legible (UTF-8)
            datos_crudos = conn.recv(1024).decode('utf-8')

            # Si .recv() retorna vacío, significa que el cliente cerró la conexión del otro lado
            if not datos_crudos:
                print(f"\n[RED] La PC ({ip_PCesp}) se ha desconectado. Esperando nueva conexión...")
                break

            # Limpia caracteres con .strip() y separa el texto por la coma
            partes = datos_crudos.strip().split(',')

            # Verifica que el paquete de datos llegó completo (Temperatura y Tendencia)
            if len(partes) == 2:
                try:
                    # Extrae el texto, lo convierte a número decimal (float)
                    temp_float = float(partes[0])

                    # Introducimos los nuevos datos en las listas globales que lee Matplotlib
                    tiempos_x.append(contador_tiempo)
                    temperaturas_y.append(temp_float)

                    # Ventana deslizante: Mantiene solo las últimas 30 muestras en pantalla.
                    # Si superamos las 30, eliminamos el dato más viejo (índice 0) de ambas listas
                    # para evitar saturar la memoria RAM.
                    if len(tiempos_x) > 30:
                        tiempos_x.pop(0)
                        temperaturas_y.pop(0)

                    contador_tiempo += 1 # Prepara el índice en el eje X para la siguiente lectura
```

```
        except Exception as e:
            # Captura y muestra cualquier error
            print(f"Error de red: {e}")
            break

# ========================================
# FUNCIÓN MATPLOTLIB (Renderizado Gráfico)
# ========================================
def actualizar_grafico(frame):
    # .clear() borra el dibujo anterior del lienzo ('ax').
    ax.clear()

    # Plotea (dibuja) los arreglos globales
    # color='red' (línea roja), marker='o' (puntos redondos en cada medición),
    # linestyle='-' (línea continua uniendo los puntos), linewidth=2 (grosor de la línea)
    ax.plot(tiempos_x, temperaturas_y, color='red', marker='o', linestyle='-', linewidth=2)

    # Configuración estética de la interfaz
    ax.set_title("Estación Tiempo Real", fontsize=14, fontweight='bold') # Título principal
    ax.set_xlabel("N° de Muestras recibidas") # Etiqueta del eje X
    ax.set_ylabel("Temperatura (°C)")         # Etiqueta del eje Y
    ax.grid(True, linestyle='--', alpha=0.7)  # Activa una cuadrícula de fondo, punteada y algo transparente

    # Lógica de auto-escalado dinámico del Eje Y
    # Si ya recibimos al menos un dato, adaptamos el "zoom" vertical
    if len(temperaturas_y) > 0:
        # Forzamos que la ventana gráfica abarque siempre 2°C por debajo de la mínima actual...
        min_y = min(temperaturas_y) - 2.0
        # ...y 2°C por encima de la máxima actual, manteniendo la curva de temperatura bien centrada.
        max_y = max(temperaturas_y) + 2.0
        ax.set_ylim(min_y, max_y)
```

```
# ========================================
# EJECUCIÓN PRINCIPAL
# ========================================

# 1. Arranca el servidor de red en un hilo secundario concurrente (Threading)
# daemon=True indica que si cerramos la ventana del gráfico (hilo principal),
# el hilo de red se debe apagar automáticamente.
hilo_red = threading.Thread(target=recibir_datos, daemon=True)
hilo_red.start()

# 2. Creación de la ventana principal y el lienzo (Axes) de Matplotlib
# fig: es la ventana
# ax: es el recuadro blanco adentro de la ventana donde se dibuja el gráfico.
# figsize=(8, 5) define el tamaño inicial de la ventana
fig, ax = plt.subplots(figsize=(8, 5))
fig.canvas.manager.set_window_title('Monitor de Temperatura TCP/IP') # Cambia el nombre de la barra de título de la ventana

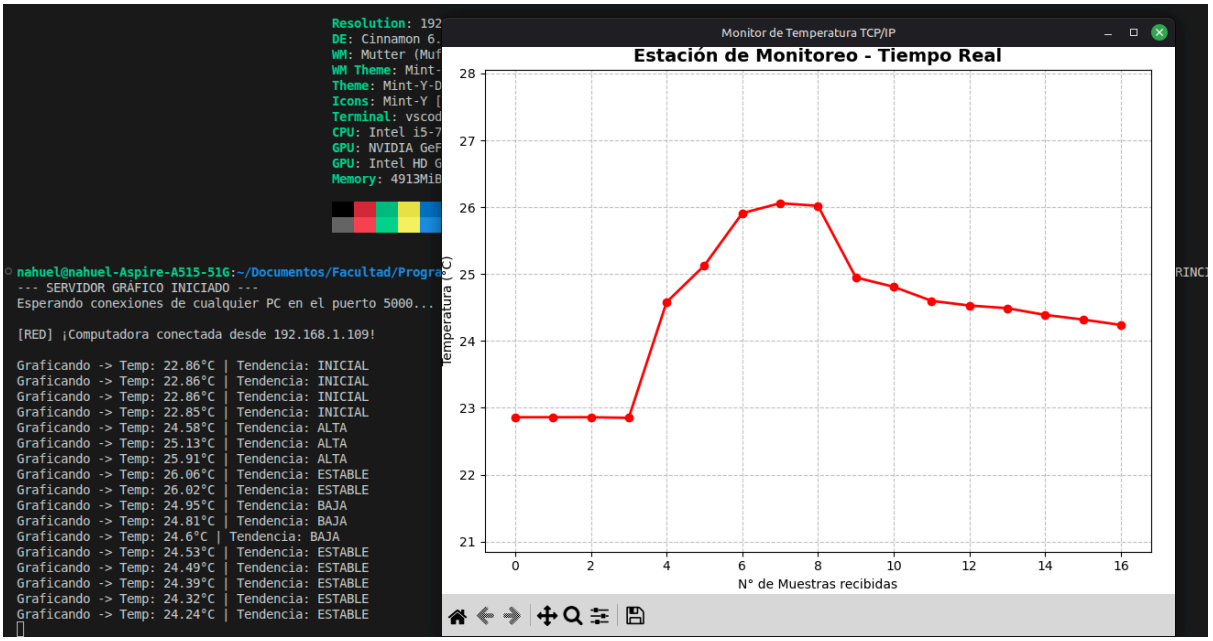
# 3. Ejecuta el bucle de renderizado infinito
# FuncAnimation llama automáticamente a nuestra función 'actualizar_grafico'
# cada 500 milisegundos (interval=500), redibujando la pantalla con los datos nuevos.
ani = animation.FuncAnimation(fig, actualizar_grafico, interval=500, cache_frame_data=False)

# 4. Ajusta automáticamente los márgenes para que los títulos no se corten
plt.tight_layout()

# 5. Muestra la ventana de la GUI (Interfaz Gráfica) y detiene el código aquí hasta que el usuario cierre la ventana
plt.show()
```

Captura de pantalla demostrando el funcionamiento

En la imagen se puede ver como se reciben los datos y son representados en el gráfico



Conclusión final

Cumplimiento de especificaciones

A lo largo del desarrollo del proyecto, se logró dar cumplimiento satisfactorio a todos los requerimientos y restricciones especificadas en cada bloque de la consigna.

Consideraciones Finales

El proyecto permitió no solo aplicar de manera práctica las herramientas y conceptos teóricos vistos durante el cursado, sino también comprender la importancia de planificar y estructurar el sistema antes de empezar su programación, ahorrando tiempo y evitando posibles errores en el futuro, como así también permitiendo la correcta comunicación entre los distintos programas realizados.